

BUSN 41210: Financial Analytics

Lecture 6: Classification

Dacheng Xiu

Chicago Booth

September 18, 2026

© 2026 Dacheng Xiu, Chicago Booth Restricted Materials. All Rights Reserved.

Outline

1. Logistic regression
2. K -nearest neighbors and group membership.
3. Binary classification: from probabilities to decisions.
4. Misclassification, sensitivity and specificity.

Logistic Regression

Linear regression is just one type of linear model.

Logistic regression: when y is true or false (1/0).

Binary response as a prediction target:

- ▶ Profit or Loss, greater or less than, Pay or Default.
- ▶ Thumbs up or down, buy or not buy, potential customer?
- ▶ Win or Lose, Sick or Healthy, Republican or Democrat.

Building a linear model for **binary response data**

Recall our original model specification: $\mathbb{E}[y \mid \mathbf{x}] = f(\mathbf{x}'\boldsymbol{\beta})$.

The response ' y ' is 0 or 1, leading to conditional mean:

$$\mathbb{E}[y|\mathbf{x}] = p(y = 1|\mathbf{x}) \times 1 + p(y = 0|\mathbf{x}) \times 0 = p(y = 1|\mathbf{x}).$$

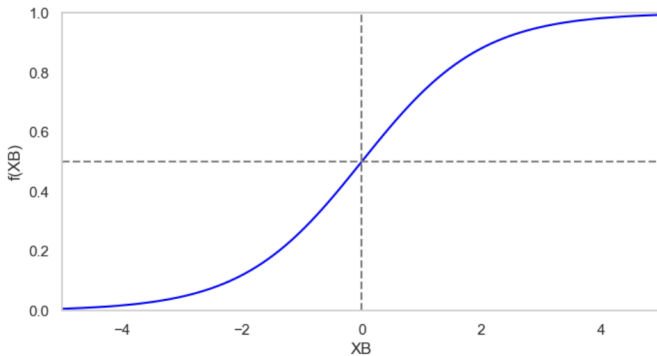
$\Rightarrow$ The expectation is a probability.

We'll choose $f(\mathbf{x}'\boldsymbol{\beta})$ to give values between zero and one.

We want a binary choice model

$$p = P(y = 1 \mid \mathbf{x}) = f(\beta_0 + \beta_1 x_1 \dots + \beta_p x_p)$$

where f is a function that increases in value from zero to one.



We'll use the logit link and do **'logistic regression'**.

$$P(y = 1|\mathbf{x}) = \frac{e^{\mathbf{x}'\boldsymbol{\beta}}}{1 + e^{\mathbf{x}'\boldsymbol{\beta}}} = \frac{\exp[\beta_0 + \beta_1 x_1 \dots + \beta_d x_d]}{1 + \exp[\beta_0 + \beta_1 x_1 \dots + \beta_d x_d]}$$

The **'logit'** link is common, for a couple of good reasons.

One big reason: A little algebra shows

$$\log \left[\frac{p}{1-p} \right] = \beta_0 + \beta_1 x_1 \dots + \beta_d x_d,$$

so that it is a linear model for log-odds.

Estimate the Probability of Default

Credit Card Clients does binary regression: **Default** versus **not Default**.

Say $y = 1$ for Default, otherwise $y = 0$.

Default.csv has for 30,000 credit card clients (about 6,636 default) related info.

Logistic regression fits $p(y = 1)$ as a function of related information.

The following link gives more details on this data set.

[Default of Credit Card Clients Data Set](#)

Logistic regression is easy in Python

Again using glm:

```
smf.glm(formula=my_formula, data=default,  
        family=sm.families.Binomial()).fit()
```

glm stands for Generalized Linear Model.

The argument 'family=sm.families.Binomial()' indicates y is binary.

The response can take a few forms:

- ▶ $y = 1, 1, 0, \dots$ numeric vector.
- ▶ $y = \text{TRUE}, \text{TRUE}, \text{FALSE}, \dots$ logical.
- ▶ $y = \text{'win'}, \text{'win'}, \text{'lose'}, \dots$ factor.

Everything else is the same as for linear regression.

Interpreting Coefficients

The model is

$$\frac{p}{1-p} = \exp[\beta_0 + x_1\beta_1 + \dots + x_p\beta_p]$$

So $\exp(\beta_j)$ is the **odds multiplier** for a unit increase in x_j .

$b['AGE'] = 0.007$, so one unit increase in AGE multiplies odds of default by $\exp(0.007) \approx 1.007$.

$b['PAY_0'] = 0.577$, so one unit increase in last month payment multiplies odds of default by $\exp(0.577) \approx 1.781$.

What is the odds multiplier for a covariate coefficient of zero?

MLE for Logistic Regression

Our logistic regression likelihood is the product

$$\begin{aligned} L(\beta) &= \prod_{i=1}^n P(y_i | \mathbf{x}_i) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} \\ &= \prod_{i=1}^n \left(\frac{\exp[\mathbf{x}_i' \beta]}{1 + \exp[\mathbf{x}_i' \beta]} \right)^{y_i} \left(\frac{1}{1 + \exp[\mathbf{x}_i' \beta]} \right)^{1-y_i} \end{aligned}$$

This is maximized by minimizing the deviance

$$\begin{aligned} Dev &= -2 \sum_{i=1}^n (y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) \\ &\propto \sum_{i=1}^n \left[\log(1 + e^{\mathbf{x}_i' \beta}) - y_i \mathbf{x}_i' \beta \right] \end{aligned}$$

We have the same output as for a linear/gaussian model.

But the 'dispersion parameter' here is always set to one.

Check this to make sure you've actually run logistic regression.

```
proba.summary()  
(Dispersion parameter for binomial family taken to be 1)  
Residual deviance: 27877  on 29976  degrees of freedom
```

'degrees of freedom' is actually 'number of observations - df', where df is the number of coefficients estimated in the model.

That is, $df(\text{deviance}) = \text{nobs} - df(\text{regression})$.

From the Python output, how many observations do we have?

Sum of Squares (Deviance) is the bit we need to minimize:

$$Dev \propto \sum_{i=1}^n (y_i - \mathbf{x}_i \beta)^2$$

This makes the observed data *as likely as possible*.

Error variance σ^2 measures variability around the mean.

i.e., $\sigma^2 = \text{var}(\varepsilon)$, where $\varepsilon_i = y_i - \mathbf{x}_i \beta$ are the 'residuals'.

$$\hat{\sigma}^2 = \frac{1}{n - p - 1} \sum \hat{\varepsilon}_i^2$$

Python estimates σ^2 and calls it the **Scale**.

Even if we know β , we only predict log sales *with uncertainty*.

e.g., there's a 95% probability of log sales in $\mathbf{x}'\beta \pm 2\sqrt{0.48}$

Residual deviance D is what we've minimized, using $\mathbf{x}'\beta$.

Null deviance D_0 is for the model where you don't use $\mathbf{x}$.

i.e., if you use $\hat{y}_i = \bar{y}$:

▶ $D_0 = \sum (y_i - \bar{y})^2$ in linear regression.

▶ $D_0 = -2 \sum [y_i \log(\bar{y}) + (1 - y_i) \log(1 - \bar{y})]$ in logistic reg.

The difference between D and D_0 is due to info in $\mathbf{x}$.

Proportion of deviance explained by $\mathbf{x}$ is called R^2 :

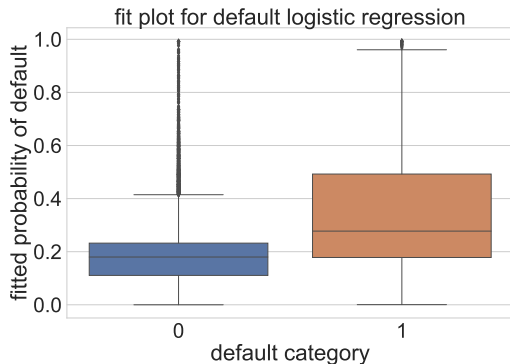
$$R^2 = \frac{D_0 - D}{D_0} = 1 - \frac{D}{D_0}.$$

This measures how much variability you are able to model.

in proba: $R^2 = 1 - 27877/31705 = 0.12$

Fit plots for logistic regression

We can plot $\hat{y}$ vs y in logistic regression using a boxplot.



The estimation pushes each distribution away from the middle.
Where would you choose for a classification cut-off?

Implementation

Linear regression has a closed-form (analytical) solution. $\hat{\beta} = (X'X)^{-1}X'y$.

Logistic regression requires a numerical procedure. For instance, we can adopt **gradient descent** of the loss function.

Gradient descent: applicable in the search of local minimum of a differentiable function. Extremely useful (with some twists) in deep learning.

Algorithm:

1. Start with an initial value β_0 .
2. Iterate

$$\beta_{t+1} = \beta_t - \gamma_t \nabla \text{loss}(\beta_t).$$

3. Stop when converged.

γ_t is called learning rate – the most important parameter to tune.

Prediction

In logistic regression, we use `reg.predict(mynewdata)` to get probabilities $e^{\mathbf{x}'\hat{\beta}}/(1 + e^{\mathbf{x}'\hat{\beta}})$:

```
proba.predict(default[0:4])
```

1	2	3	4
0.505544	0.150024	0.200426	0.242773

`newdata` *must* match the format of original data.

Out-of-Sample Prediction

You care about how your model predicts out-of-sample (OOS).

One way to test this is to use a validation sample.

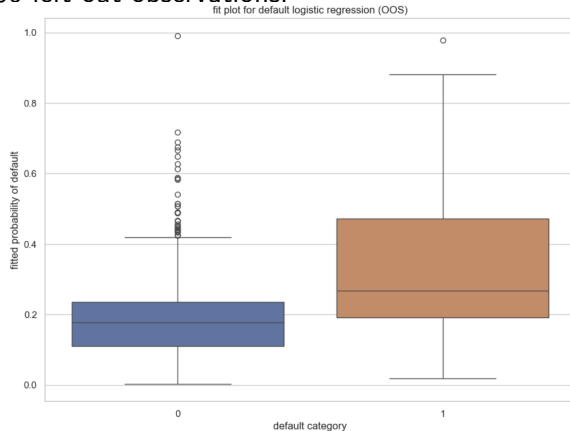
Fit your model to the remaining *training data*,
and see how well it predicts the *left-out data*.

```
# Sample 1000 random indices
leaveout = sample(range(len(default)),1000)
# train the model WITHOUT these observations
probatrain = smf.glm(formula=my_formula ,
                     data=default.drop(leaveout), family=sm.families.Binomial()).fit()

# predicted probability of default on the left out data
pdefault = probatrain.predict(default.iloc[leaveout])
```

Out-of-Sample Prediction

Fit plots on the 1000 left out observations.



For the left-out data, we get $D0 = 1063.897$, $D = 934.8424$, $R^2 = 0.1213$.
Since the sample is random, you might get different results.

Classification

Just as in linear regression, we have a set of training observations $(x_1, y_1), \dots, (x_n, y_n)$. But now y_i are qualitative rather than quantitative, i.e. y is membership in a category $\{1, 2, \dots, m\}$.

The classification problem: given new x_i^{new} what is the class label $f(x_i^{new})$?

The quality of classifier can be assessed by its missclassification risk, i.e. probability of falsely classifying a new observation

$$P(Y^{new} \neq f(x_i^{new}))$$

This quantity is unknown but can be estimated by a proportion of wrong labels in a validation dataset. Good classifiers yield small risk.

Bayes Classifier

There is actually a theoretically optimal classifier, the *Bayes classifier*, which minimizes the misclassification risk.

The idea is to assign each observation to *the most likely class, given its predictor values*, i.e. choose the class $j \in \{1, 2, \dots, m\}$ for which

$$P(Y = j|X = x)$$

is the largest.

Unfortunately $P(Y = j|X = x)$ is not known. Bayes classifier is **unattainable gold standard**. But! We can estimate it!

Analogy to Regression

	Classification	Regression
Risk	classification risk $P(Y \neq f(X))$	squared error risk $E(Y - f(X))^2$
Minimizer	Bayes classifier $f(x) = \arg \max_j P(Y = j X = x)$	conditional expectation $f(x) = E(Y X = x)$

Classifiers

There are many ways to estimate $P(Y = j|X = x)$ from the training data.

► We can go *parametric*:

- Assume that $P(Y = j|X = x, \beta)$ is a specific function of unknown parameters β and learn those.
- Sounds familiar? Logistic regression

► We can go *non-parametric*:

- We estimate $P(Y = j|X = x)$ directly without estimating any parameters.
- K-nearest Neighbors (KNN)

Nearest Neighbors

The idea is to estimate $P(Y = j|X = x_{new})$ locally by looking at the labels of similar observations that we already saw.

KNN: what is the most common class around x ?

1. Take K-nearest neighbors $\mathbf{x}_{i_1} \dots \mathbf{x}_{i_k}$ of x_{new} in the training data

Nearness is in euclidean distance: $\sqrt{\sum_{j=1}^p (x_j - x_{ij})^2}$.

2. Estimate

$$\hat{P}(Y = j|X = x_{new}) = \frac{1}{K} \sum_{k=1}^K \mathbf{1}_{\{y_{i_k}=j\}}.$$

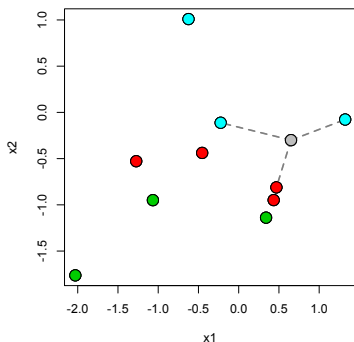
3. Select the class with highest $\hat{P}(Y = j|X = x_{new})$ (Bayes classifier).

Since we're calculating distances on X , scale Matters!

We'll use Python's StandardScaler function in sklearn to divide each x_j by $\text{sd}(x_j)$

The new units of distance are in *standard deviations*.

Nearest Neighbors



K -NN's collaborative estimation:

Each neighbor votes.

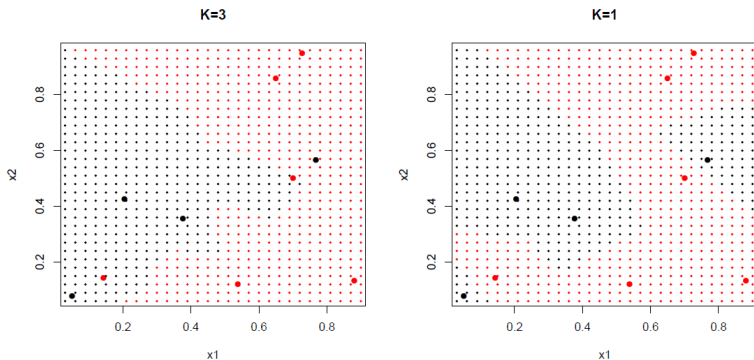
Neighborhood is by shortest distance (shown as the dashed lines).

The relative vote counts provide a **very crude** estimate of probability.

For 3-nn, $p(\text{blue}) = 2/3$, but for 4-nn or 2-nn, it's only $1/2$.

Sensitive to neighborhood size (think about extremes: 1 or n).

Nearest Neighbors: Decision Boundaries



- ▶ Larger K leads to higher training error (proportion of in-sample misclassification rate)
- ▶ Smaller K leads to higher flexibility (overfitting and poor out-of-sample misclassification rate)

Nearest Neighbors in Python (Implementation)

Compute the euclidean distance of X_{new} in the training data

```
def euc_distance(arr1, arr2):  
    distance = np.sqrt(sum((arr1-arr2)**2))  
    return distance
```

Sort by distance, find the index of the closest K neighbors and select the class with highest possibility.

```
def knn_classifier(X, y, testVector, k):  
    distance_list = [euc_distance(testVector, x) for x in X]  
    neighbors = np.argsort(distance_list)[:k]  
    count = Counter(y[neighbors][0])  
    return count.most_common()[0][0]
```

Nearest Neighbors in Python (Package)

Alternatively, you may use some existing package, which is less transparent but offers more features. The `Scikit-learn` package includes function `KNeighborsClassifier`.

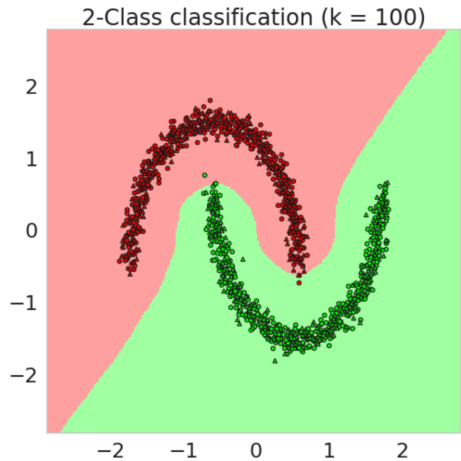
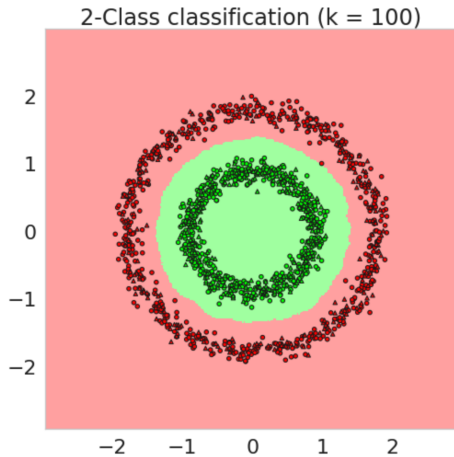
You set `n_neighbors` to specify how many neighbors get to vote.

```
from sklearn.neighbors import KNeighborsClassifier

nn1 = KNeighborsClassifier(n_neighbors=1, n_jobs=-1).fit(X_train, y_train)
nn5 = KNeighborsClassifier(n_neighbors=5, n_jobs=-1).fit(X_train, y_train)
y_pred_1 = nn1.predict(X_test)
y_pred_5 = nn5.predict(X_test)
```

You set `n_jobs` to use parallel computing.

KNN with simulated data



Lending Club Data (Revisit HW 1)

LendingClub (LC) is a US peer-to-peer lending company. It was the first peer-to-peer (P2P) lender to register its offerings as securities with the Securities and Exchange Commission (SEC), and to offer loan trading on a secondary market.

In this case study, we understand how data is used to minimize the risk of losing money while lending to customers in P2P business.

We consider 2 possible loan status

Fully paid(0): Applicant has fully paid the loan

Charged-off(1): Applicant has defaulted on the loan

We have 27 explanatory variables, 15 of which are categorical variables.

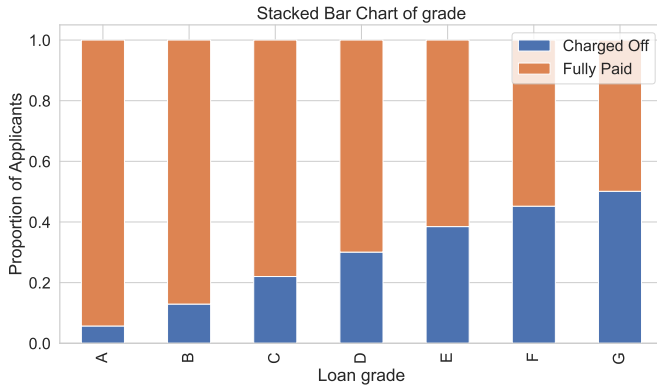
We use loans issued in 2012–2018 as the training sample and loans issued in 2019–2020 as the test sample.

Lending Club Data: Training and Test Samples

	training sample	test sample
loans issued	Mar 2012 – Dec 2018	Jan 2019 – Sep 2020
number of loans	1,199,593	79,391
default rate	19.6%	17.0%

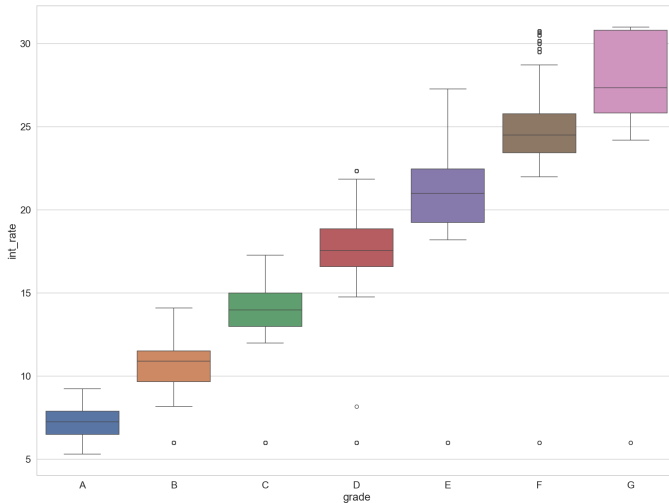
- ▶ The split is by **issue date**: every loan in the training sample was issued before every loan in the test sample, as it would be if the model were put to use.
- ▶ Missing values are filled with **training-sample** medians, and the dummy variables are defined on the training sample; the same transformation is then applied to the test sample.
- ▶ We leave out LC's own `grade`, `sub_grade` and `int_rate` when we build our models, and use them to compare our predictions with LC's assessment.

Some Variables are Special: grade



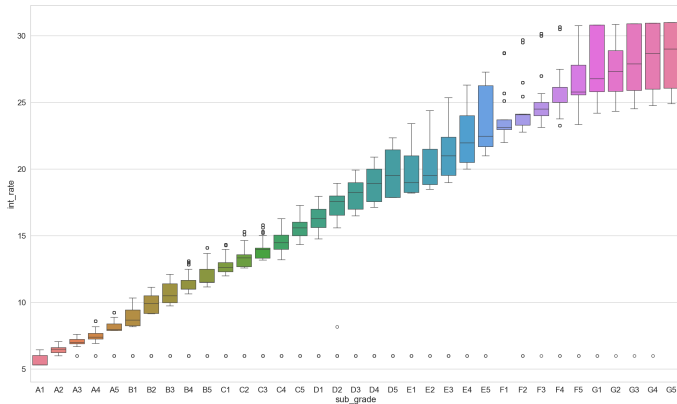
As grade varies from A to G, the default probability increases.

Some Variables are Special: interest rate vs grade



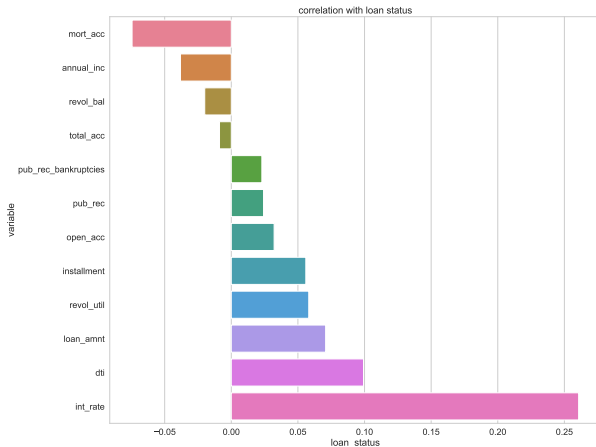
As default probability increases, interest rate increases. Both are determined by LC.

Some Variables are Special: interest rate vs subgrade



The same is true for subgrade. We will use these variables to compare with LC's model.

Correlations with Default

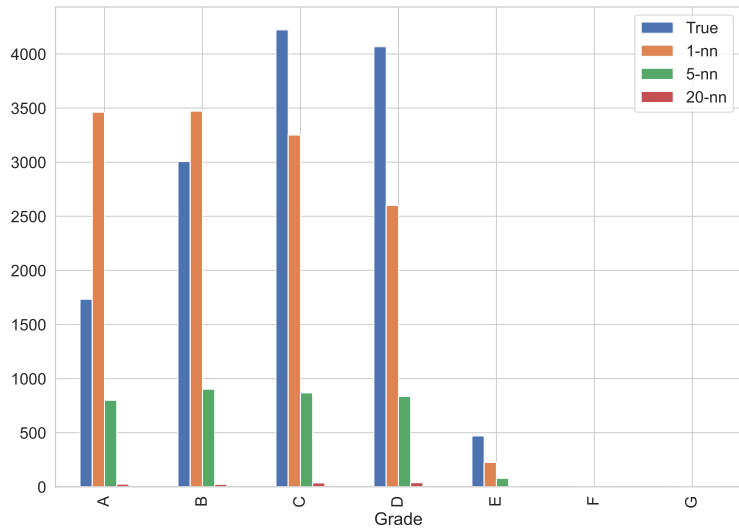


Some covariates are better **discriminators** (dti, mort_acc).

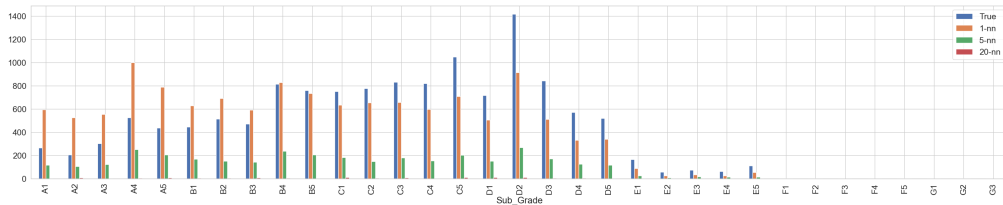
mort_acc: number of mortgage accounts

dti: borrower's debt-to-income ratio

Predicted # of Defaults by Grade in Testing Sample



Predicted # of Defaults by Subgrade in Testing Sample



5-nn and 20-nn clearly underestimate # of default, but in terms of accuracy, they dominate 1-nn!

Accuracy ($P(\hat{Y} = Y)$): 1-nn: 0.72, 5-nn: 0.80, 20-nn: 0.83

KNN: Pros and Cons

Pros

- ▶ KNN's are simple
- ▶ KNN's naturally handle multiple categories ($m > 2$)
- ▶ KNN's will outperform linear classifiers when the decision boundary is non-linear

Cons

- ▶ Computing neighbors can be costly for large n and p .
- ▶ KNN's do not perform variable selection. How did we avoid the curse of dimensionality?
- ▶ Choosing K can be tricky. Cross-validation works, but is unstable.
- ▶ And the classification is very sensitive to K .
- ▶ All you get is a classification, with only rough local probabilities. Without good probabilities we cannot assess uncertainty.
- ▶ There is no natural or simple way to handle categorical variables.

Binary Classification via Logistic Regression

Many decisions can be reduced to binary classification. $y_i \in \{0, 1\}$

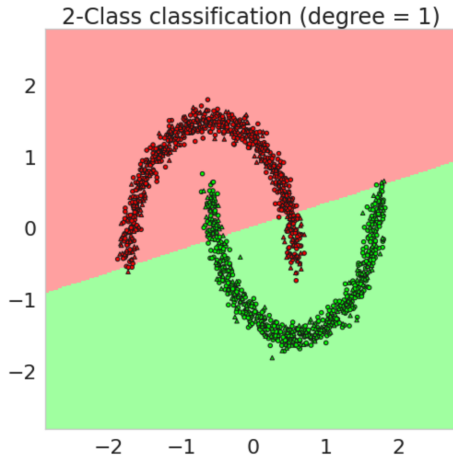
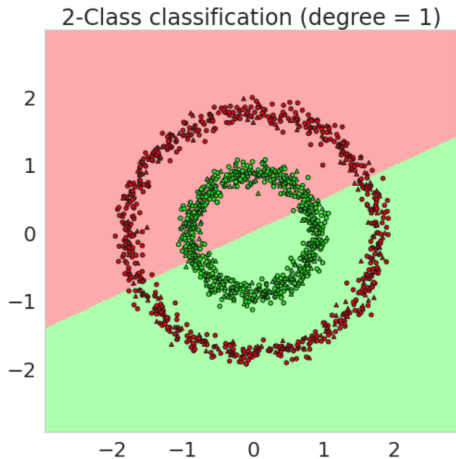
KNN's were an example of a *non-parametric* classification method.

A useful parametric alternative for two categories is the *logistic regression*.

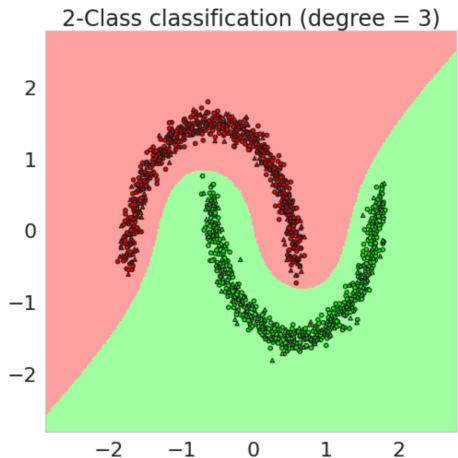
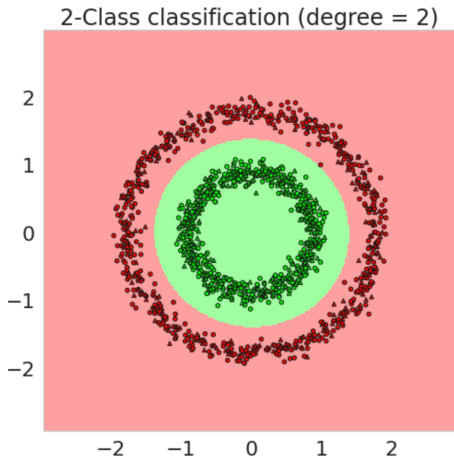
Compared to KNN's

- ▶ *Logistic regression* yields parametric decision boundaries (linear, quadratic depending on our regression equation) $\implies$ it is principled but it can be flexible
- ▶ *Logistic regression* is a 'global' method, i.e. it uses all the training data to estimate probabilities, not just neighbors $\implies$ probability estimates are more stable
- ▶ *Logistic regression* can do variable selection! (yay!) Note that `Scikit-learn's LogisticRegression` function utilizes Ridge penalty by default; we set `penalty=None` to obtain the MLE.

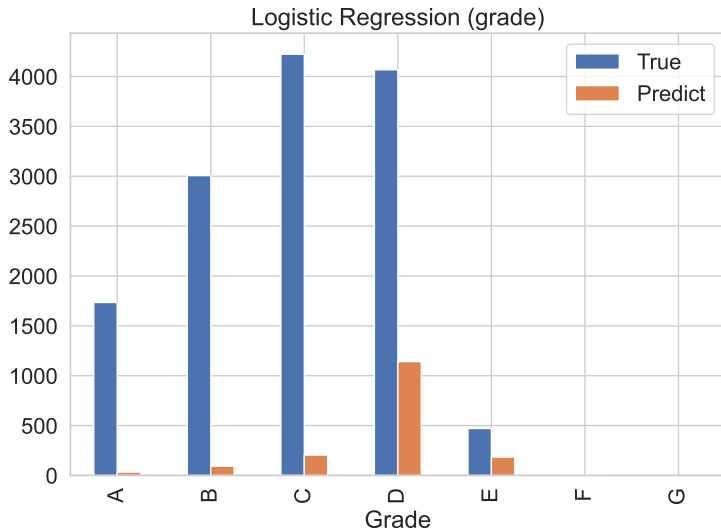
Logistic Regression with Simulated Data



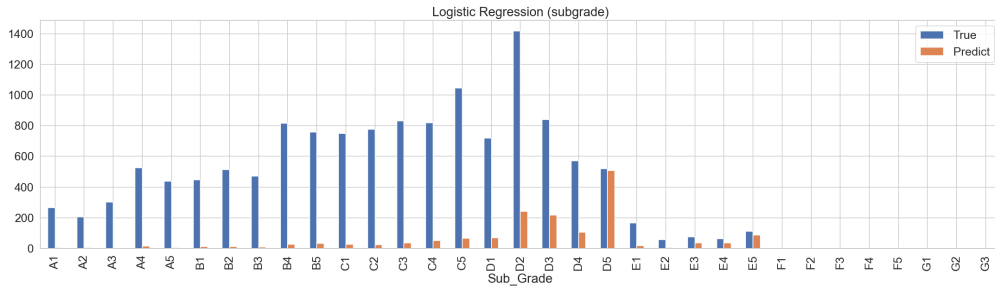
Logistic Regression with Simulated Data



Logistic Regression



Logistic Regression



Accuracy ($P(\hat{Y} = Y)$): Logit: 0.82

Decision Making

Logistic regression gives us an estimate of $P(Y = 1|X = x, \beta)$.

The Bayes decision rule is based purely on probabilities: classify as a defaulter, i.e., $\hat{Y} = 1$, when

$$P(Y = 1|X = x, \hat{\beta}) > 0.5.$$

Recall that the Bayes rule is optimal in terms of **misclassification error** $P(\hat{Y} \neq Y)$.

Decision Making

There are two ways to be wrong in a binary problem.

$$P(\hat{Y} \neq Y) = P(\hat{Y} = 1|Y = 0)P(Y = 0) + P(\hat{Y} = 0|Y = 1)P(Y = 1).$$

- ▶ False **positive** (Type I error): predict $\hat{Y} = 1$ when $Y = 0$.
(classify as defaulters when they are not)
- ▶ False **negative** (Type II error): predict $\hat{Y} = 0$ when $Y = 1$.
(classify as non-defaulters when they in fact are)

Misclassification error weights different errors by the relative weights of different labels in the population.

Asymmetric Error Importance and Imbalanced Class Proportion

Both false positives and false negatives are bad, but sometimes one of them can be much worse $\implies$ the cost *can be asymmetric*!

- ▶ investment
- ▶ medical diagnosis

Maximizing accuracy does not appear to be the most relevant criterion in practice.

In many scenarios in practice, the class label for worse outcome is also rare!

- ▶ fraud detection
- ▶ default prediction
- ▶ rare disease

Maximizing accuracy puts more weight on the larger class. $P(Y = 0)/P(Y = 1)$ is large $\implies$ far more false negatives!

Minimizing Cost

Say that, on average, for every 1\$ loaned lenders make 25¢ in interest if it is repayed but lose 1\$ if they default.

This gives the following action-cost matrix

	<i>payer</i>	<i>defaulter</i>
<i>loan</i>	-0.25	1
<i>no loan</i>	0	0

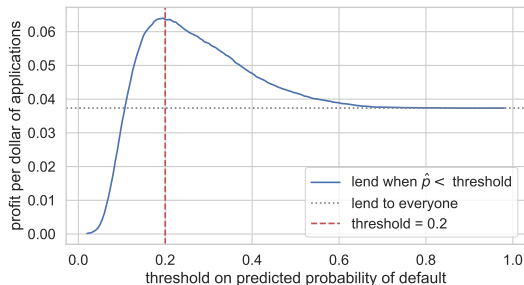
Suppose you estimate p for the probability of default.

Expected *profit* from lending is greater than zero if

$$(1 - p)\frac{1}{4} - p > 0 \Leftrightarrow \frac{1}{4} > \frac{5}{4}p \Leftrightarrow p < 1/5$$

From this simple matrix you should lend whenever probability of default is less than 0.2.

Minimizing Cost: Lending Club



rule	profit per \$	funded
lend to everyone	0.037	100%
lend when $\hat{p} < 0.5$	0.042	98%
lend when $\hat{p} < 0.2$	0.064	67%

Lend when the predicted probability of default is below a threshold. Profit is 25¢ per dollar repaid and $-1\$$ per dollar defaulted, computed in the test sample.

The best threshold in the test sample is 0.195 — essentially the 1/5 implied by the cost matrix.

Confusion Matrix

An alternative approach is to think about False/True Positive Rates in tandem when evaluating a given classification rule.

We can use a confusion matrix to show information about actual and predicted classifications done by a classifier.

		Predicted	
		Negative (N) -	Positive (P) +
Actual	Negative -	True Negatives (TN)	False Positives (FP) Type I error
	Positive +	False Negatives (FN) Type II error	True Positives (TP)

Confusion Matrix in Python

Load the `Scikit-learn` package which includes function `confusion_matrix`.

`y_test` and `y_pred` are true values and predicted values.

```
from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred)
```

5-nn

```
array([[63116, 2769],
       [12791, 715]])
```

20-nn

```
array([[65792, 93],
       [13473, 33]])
```

logit

```
array([[64846, 1039],
       [12889, 617]])
```

FP and FN Rates

False Positive Rate: # misclassified as pos / # actual neg. ($\frac{FP}{FP+TN}$)

False Negative Rate: # misclassified as neg / # actual pos. ($\frac{FN}{FN+TP}$)

False Positive Rate

FPR = FP/(FP+TN) 5-nn: 0.042 20-nn: 0.001 logit: 0.016

False Negative Rate

FNR = FN/(FN+TP) 5-nn: 0.947 20-nn: 0.998 logit: 0.954

We can choose a threshold to control false positive rate, while minimizing false negative rate.

Sensitivity and Specificity

Two more common classification rates are

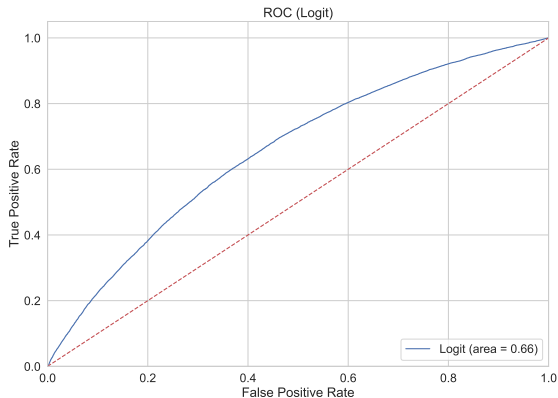
sensitivity: proportion of true $y = 1$ classified as such. (aka True Positive Rate)

specificity: proportion of true $y = 0$ classified as such. (aka True Negative Rate)

A rule is **sensitive** if it predicts 1 for most $y = 1$ observations, and **specific** if it predicts 0 for most $y = 0$ observations.

All our rules are similarly specific, though not sensitive at all. This has to do with the fact that the majority class (83% of the test sample) is non-defaulters.

The ROC Curve: Sensitivity vs 1-Specificity



Receiver Operating Characteristic: performance measure of a classifier as cutoff varies
A tight fit has the curve forced into the top-left corner.

AUC

The Area Under the Curve (AUC) is the measure of the ability of a classifier to distinguish between classes and is used as a summary of the ROC curve.

- ▶ $AUC = 1$: perfect separation.
- ▶ The higher the AUC, the better the classifier.
- ▶ $0.5 < AUC < 1$: acceptable.
- ▶ $AUC = 0.5$: either random guessing or constant for all.
- ▶ $AUC = 0$: all predictions are off.

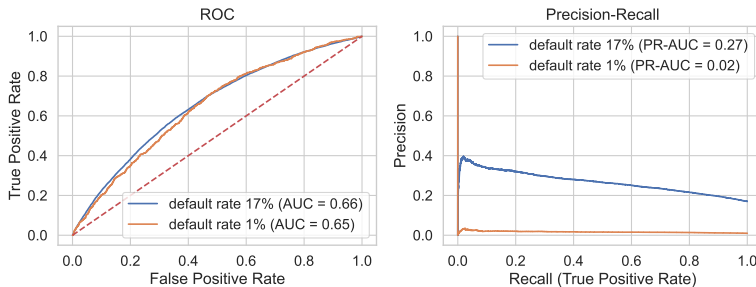
AUC: Lending Club

The logistic regression is estimated on loans issued in 2012–2018 and evaluated on loans issued in 2019–2020.

	training sample	test sample
AUC	0.70	0.66

- ▶ The test-sample AUC is the relevant one: the model is used on loans issued **after** the ones it was estimated on.
- ▶ Performance is lower out of sample than in sample, as we have seen with regression.
- ▶ For comparison, a logistic regression on `mort_acc` and `dti` alone has a test-sample AUC of 0.58.

ROC and Precision-Recall Curves with Rare Events



Precision: proportion of predicted $\hat{Y} = 1$ that are true $y = 1$. The PR curve plots precision against sensitivity (recall) as the cutoff varies.

Same model, same non-defaulters, but keep only enough defaulters for a 1% default rate:

- ▶ AUC is unchanged (0.66): sensitivity and specificity do not depend on the class proportions.
- ▶ The area under the PR curve falls from 0.27 to 0.02: precision does.

With rare events (fraud, default), report precision and recall in addition to the ROC curve.

Precision at k and Lift

Rank the test-sample loans by predicted probability of default and look at the riskiest $k\%$.

riskiest $k\%$	loans	default rate	lift
1%	793	0.39	2.3
5%	3,969	0.34	2.0
10%	7,939	0.32	1.9
20%	15,878	0.29	1.7
50%	39,695	0.24	1.4

Lift is the default rate among the selected loans relative to the default rate in the whole test sample (0.17). It is a useful summary when only a fixed number of cases can be reviewed.

Summary: Reporting a Classifier

When you report a classifier, include:

- ▶ **the question and the target**: what decision the model informs, and what exactly is predicted;
- ▶ **the data and the split**: the training and test samples, with dates and sample sizes;
- ▶ **the model**: the estimator and how any tuning parameter was chosen;
- ▶ **performance in the test sample** against a benchmark: accuracy with the majority-class rate next to it; AUC, and precision and recall when the event is rare;
- ▶ **the decision rule**: the threshold, and the costs that justify it.

The final project asks for a one-page summary of this kind for the model you submit; the template arrives with the project.